

# Creating Song List II

Week 06.



SOFTWARE CLUSTER

// COMT • **COMPUTATIONAL THINKING** (CIT1C18)



# Overview

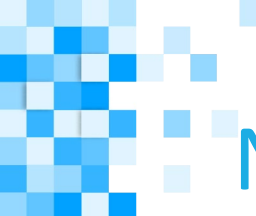
- In this workshop, you will learn how to create the “Song” class and create “Song” objects using the class.



# Objectives

---

- Demonstrate how to create object class;
- Demonstrate how to use Object Oriented manner to create songs;



## Note

- Throughout this document, the acronym “**app**” will be used to refer to **mobile application**.

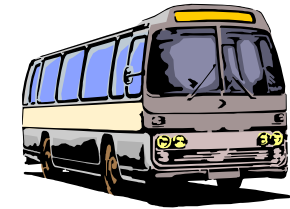
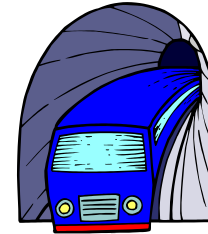


# CREATE A SONG OBJECT

# Object-Oriented Programming

- A particular style of programming that mimics real world objects is known as **Object-Oriented Programming**.
- This style of programming has been widely used in the industry for decades and provides a more natural way to store and retrieve information.
- In the next few lessons, you will learn how to create a Song object and use it in your app.

❖ Train, Bus

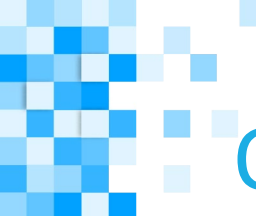


❖ Computer, Book



❖ Cat, Dog





# Object-Oriented Programming

- In the real world, we can recognise a table to be a Table; and a boy to be a Boy.
- However, how do we teach a computer to recognise what is a Table and what is a Boy?
- In terms of programming, we write **classes** to represent the objects so as to let the computer understand what the object is.
- For example, we write a **Boy class** to represent a boy virtually in our program; we write a **Table class** to represent a table virtually in our program.
- Not all programming languages support Object-Oriented Programming.
- Some languages that **do not support** are C and Pascal.
- Some languages that **support** are:
  - Java
  - C++
  - C#
  - Python
  - PHP
  - JavaScript

# Object-Oriented Programming

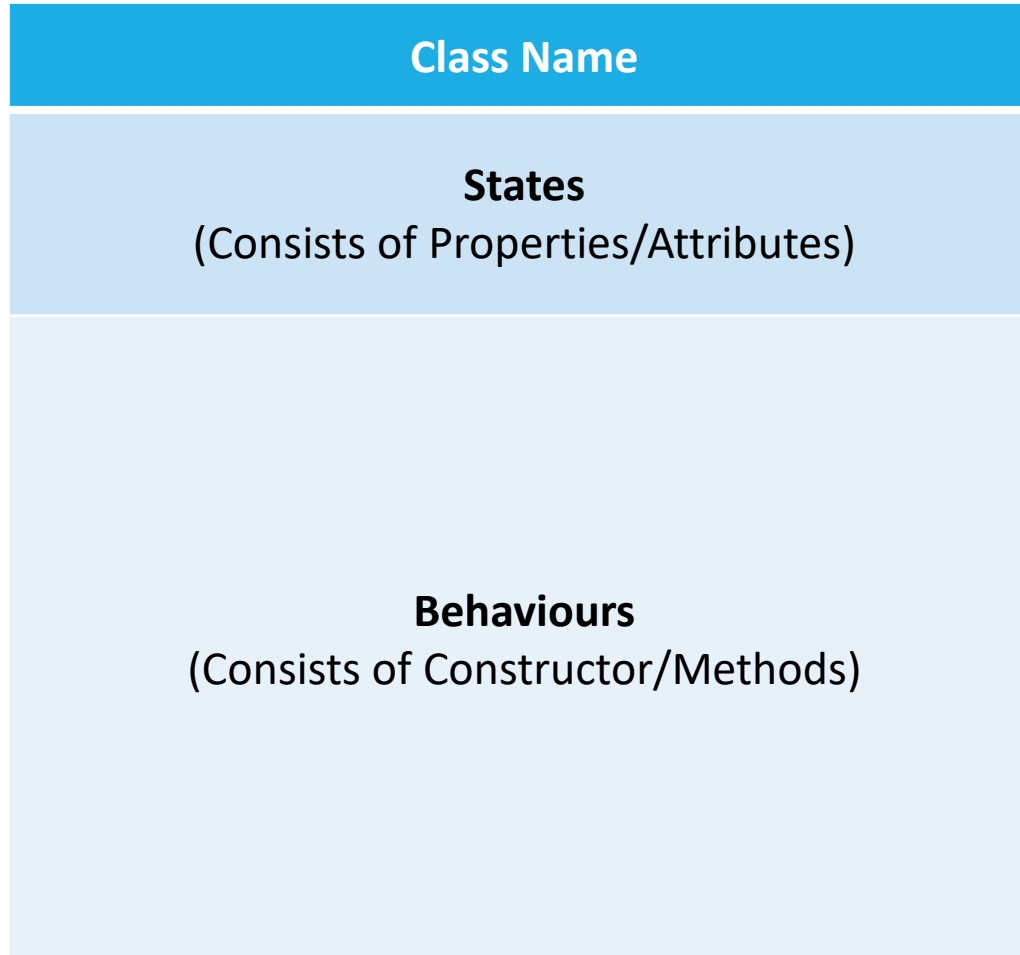
- So how does a **class** look like in Java?
- On the right is a sample of a **Boy** class.
- A **class** is a blueprint of an object.
- Every object is represented by a different class.
- Refer to next slide for more explanations.

```
1 public class Boy
2 {
3     String name;
4     String hairColour;
5     double height;
6     int age;
7
8     public Boy(n, h1, h2, a)
9     {
10         this.name = n;
11         this.hairColour = h1;
12         this.height = h2;
13         this.age = a;
14     }
15
16     public String getName()
17     {
18         return this.name;
19     }
20 }
```

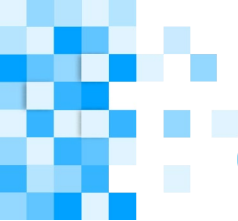


# Object-Oriented Programming

An object has 3 parts as shown here.



```
1 public class Boy
2 {
3     String name;
4     String hairColour;
5     double height;
6     int age;
7
8     public Boy(n, h1, h2, a)
9     {
10         this.name = n;
11         this.hairColour = h1;
12         this.height = h2;
13         this.age = a;
14     }
15
16     public String getName()
17     {
18         return this.name;
19     }
20 }
```



# Object-Oriented Programming

- The **States** part contains properties/attributes of an object.
- It is up to you how detailed you want the object to be.
- The more detailed you want the object to be created, the more properties/attributes you will add.
- For example, if you want to store more info about a Boy, you can add say, **skinColour**, **eyeColour**, **hairStyle**, etc to the properties.
- The **Behaviours** part contains methods that provide info of the object, or methods that change the info of the object.
- For example, to get the name of a Boy, we can create a method called **getName ()** to retrieve the name.

# Object-Oriented Programming

- After we have created a class, it is still a blueprint if we don't use it.
- For example, if we want to create a boy called John and store the info about him, we will use the class by writing the code as shown on the right.
- Note that the info/values match the parameters in the constructor of the Boy class.

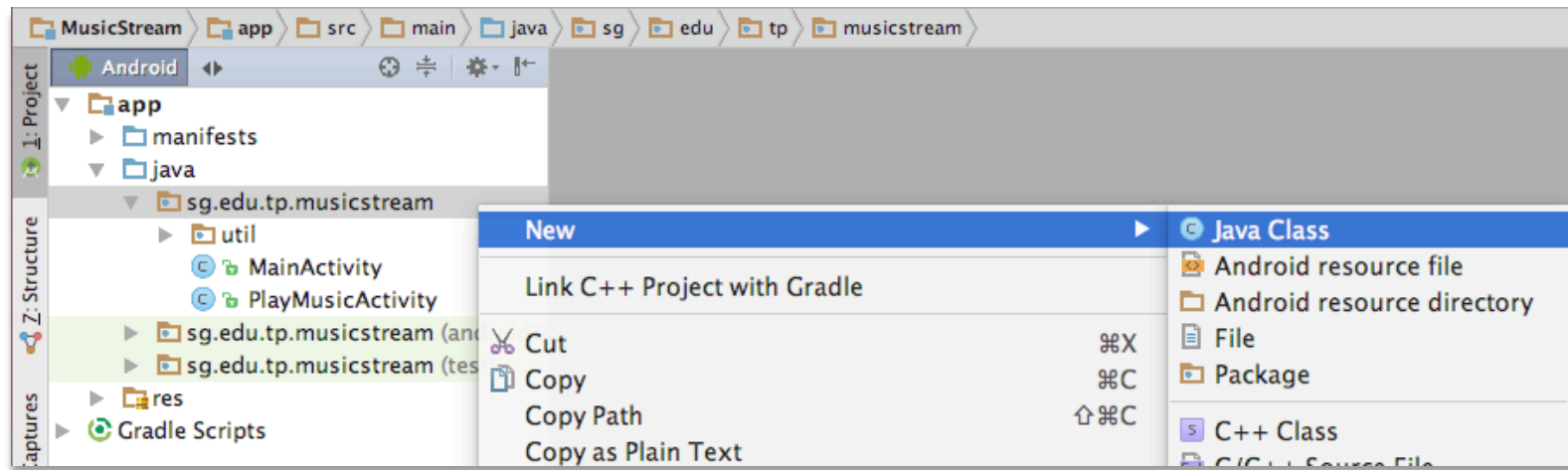
```
1 Boy john = new Boy("John", "black", 1.05, 4);
```

```
public Boy(n, h1, h2, a)
{
    this.name = n;
    this.hairColour = h1;
    this.height = h2;
    this.age = a;
}
```

A diagram with four blue arrows pointing from the arguments in the code snippet to the constructor parameters. The first arrow points from "John" to "n". The second arrow points from "black" to "h1". The third arrow points from "1.05" to "h2". The fourth arrow points from "4" to "a".

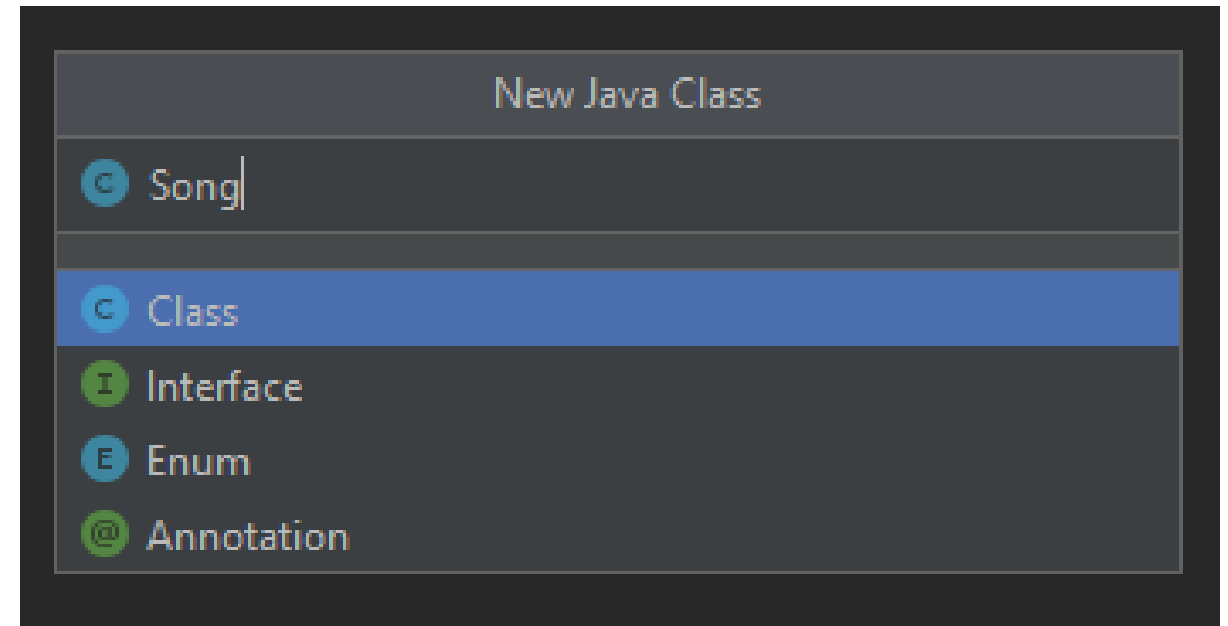
# Creating a Song Object

- The way to create a Song object in your app is by creating a Class.
- A Class is a representation of an object in code form. It is also known as a blueprint for an object.
- You can create different classes for different objects such as a bird, a plane, a phone, etc...
- To create a class, right-click on the package folder `sg.edu.tp.musicstream` and select a new Java Class as shown below.



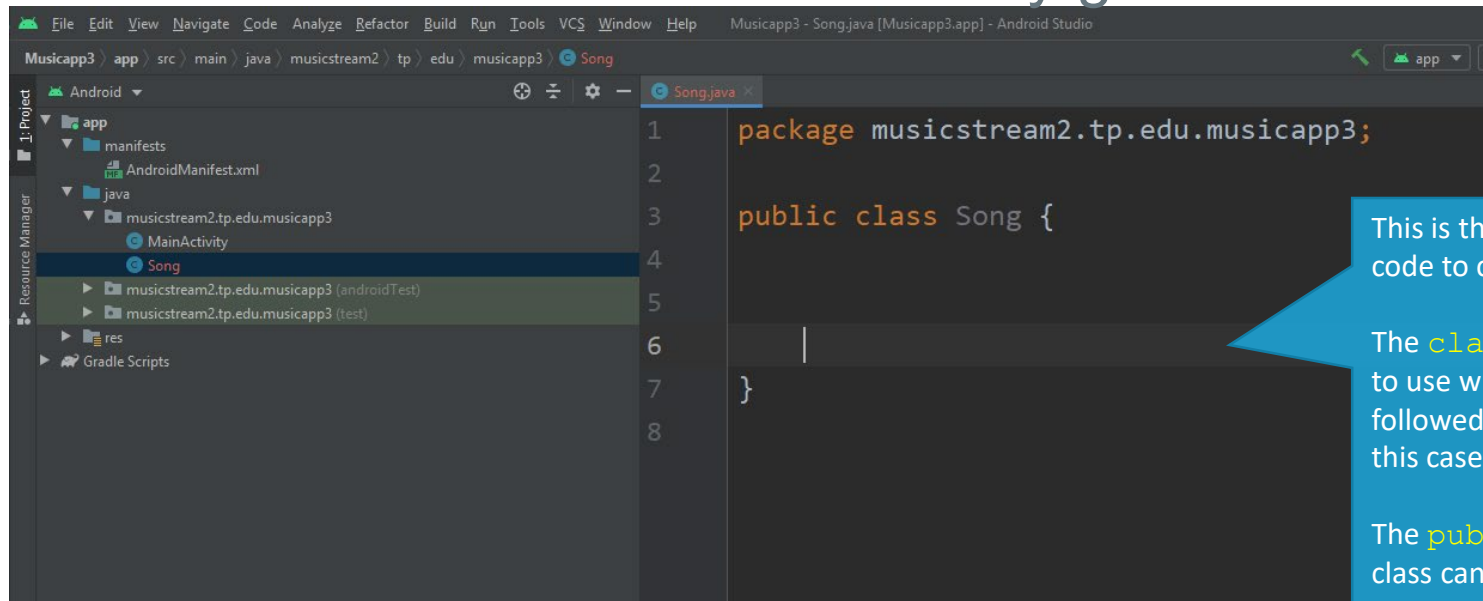
# Creating a Song Object

- In the popup window, enter `Song` as the **Name** of the class.
- All class names **MUST** start with upper case letter.
- Press Enter



# Creating a Song Object

- Once the class is created, you will see a new Java file called **Song** being created within the package folder **sg.edu.tp.musicstream**. (It is ok if your package name is different)
- A few lines of code are also automatically generated as shown below.



The screenshot shows the Android Studio interface. On the left, the 'Project' view displays the file structure of the 'Musicapp3' project, with the 'Song' class highlighted under the 'musicstream2.tp.edu.musicapp3' package. The main editor window shows the 'Song.java' file with the following code:

```
1 package musicstream2.tp.edu.musicapp3;
2
3 public class Song {
4
5
6
7 }
8
```

This is the standard starting line of code to create a class of an object.

The **class** word is a standard name to use when creating an object followed by the object name, which in this case is **Song**.

The **public** word indicates that this class can be used by other classes.

# Creating a Song Object

- Next, you will create the variables that you want to store the following items:
  - Song ID
  - Song Title
  - Artist
  - File Link
  - Song Length
  - Cover Art
- Use a name for the variable that relate closest to what you want to store. You can use the example on the right.
- For each variable, you need to provide a correct data type of the item that you want to store, such as `String` for `artist`.
- These variables are also known as the properties of the class/object.

This keyword `private` means that the variables can only be used within this class.

```
Song.java x
1 package musicstream2.tp.edu.musicapp3;
2
3 public class Song {
4
5     private String id;
6     private String title;
7     private String artiste;
8     private String fileLink;
9     private double songLength;
10    private int drawable;
11
12 }
```

This “drawable” variable is meant to store the integer of the images. Every image in Android studio such as the `billie_jean.jpg` in the drawable folder is given an integer value by android OS

# Creating a Song Object

- After creating the properties, you need to create a section of codes called the **Constructor** as highlighted below.

```
1 package musicstream2.tp.edu.musicapp3;
2
3 public class Song {
4
5     private String id;
6     private String title;
7     private String artiste;
8     private String fileLink;
9     private double songLength;
10    private int drawable;
11
12    public Song(String id, String title, String artiste, String fileLink, double songLength, int drawable){
13        this.id = id;
14        this.title = title;
15        this.artiste = artiste;
16        this.fileLink = fileLink;
17        this.songLength = songLength;
18        this.drawable = drawable;
19    }
20 }
21
```





# What is a Constructor?

- The **Constructor** is written in the form of a method, except that it DOES NOT have a return type.
- It is first executed automatically when the class is being used. Hence, its main purpose is to initialize the object.
- The name of the **Constructor MUST** be the same as the Class name, even the case **MUST** be the same.
- There are two types of Constructor:
  - Default Constructor
  - Parameterized Constructor

# What is a Constructor?

```
public Song()  
{  
    System.out.println("This is a default constructor.");  
}
```

Default constructor has no parameters, and provides the default values to the properties of the object such as `0.0` for `songLength`, `null` for `title`, etc... depending on the type of the property.

```
public Song(String id, String title, String artiste, String fileLink, double songLength, int drawable){  
    this.id = id;  
    this.title = title;  
    this.artiste = artiste;  
    this.fileLink = fileLink;  
    this.songLength = songLength;  
    this.drawable = drawable;  
}
```

Parameterized constructor is used to provide values to the properties of distinct objects. We will provide an example of this later.

# Creating a Song Object

- You can include either one or both constructors in the same class.
- In this exercise, you just need to include the parameterized constructor as shown below.

```
1 package musicstream2.tp.edu.musicapp3;
2
3 public class Song {
4
5     private String id;
6     private String title;
7     private String artiste;
8     private String fileLink;
9     private double songLength;
10    private int drawable;
11
12    public Song(String id, String title, String artiste, String fileLink, double songLength, int drawable){
13        this.id = id;
14        this.title = title;
15        this.artiste = artiste;
16        this.fileLink = fileLink;
17        this.songLength = songLength;
18        this.drawable = drawable;
19    }
20 }
21
```

Notice that all the parameters here are actually the properties of this object.

# Creating a Song Object

- The final step is to create methods for each property to set and retrieve its value.
- In the example below, two sets of the methods are created for `id` and `title` properties. Add them to your codes.

```
1 package sg.edu.tp.musicstream;
2
3 public class Song {
4     private String id;
5     private String title;
6     private String artiste;
7     private String fileLink;
8     private double songLength;
9     private int drawable;
10
11     //create a constructor for Song class
12     public Song(String id, String title, String artiste, String fileLink, double songLength, int drawable){
13         this.id = id;
14         this.title = title;
15         this.artiste = artiste;
16         this.fileLink = fileLink;
17         this.songLength = songLength;
18         this.drawable = drawable;
19     }
20
21     public String getId() { return id; }
22     public String getTitle() { return title; }
23     public String getArtiste() { return artiste; }
24     public String getFileLink() { return fileLink; }
25     public double getSongLength() { return songLength; }
26     public int getDrawable() { return drawable; }
27 }
```

The `getId` method is used to retrieve the value of the `id` of a song.

The `setId` method is used to set a new value to the `id` of the song.

Similar explanation goes for `getTitle` and `setTitle`.

The `this` keyword refers to the class itself. One of the usages for this is to differentiate variables with the same name. The variables must refer to the same property.



# Try It Yourself!

- By following the previous examples, add the set and get methods yourself for the rest of the properties:
  - Artist
  - File Link
  - Song Length
  - drawable
- Validate your finished codes with your lecturer.



# Summary

---

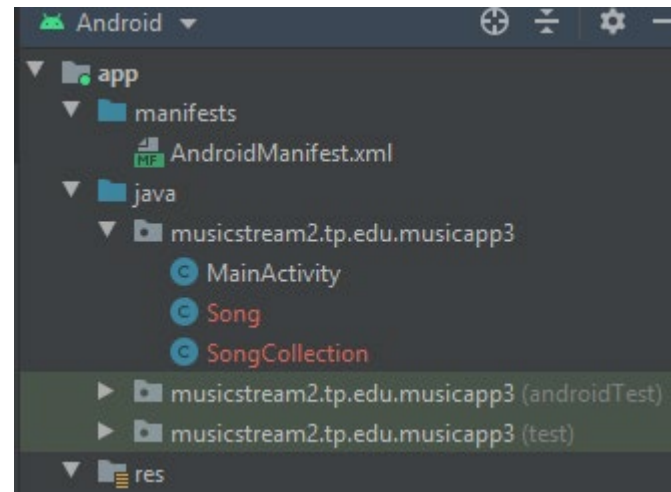
- You have now successfully created a Song object!
- In the next lesson, you will learn how to create a Song Collection object that provides song searching functionalities.



# CREATE A SONG COLLECTION OBJECT

# Creating a Song Collection Object

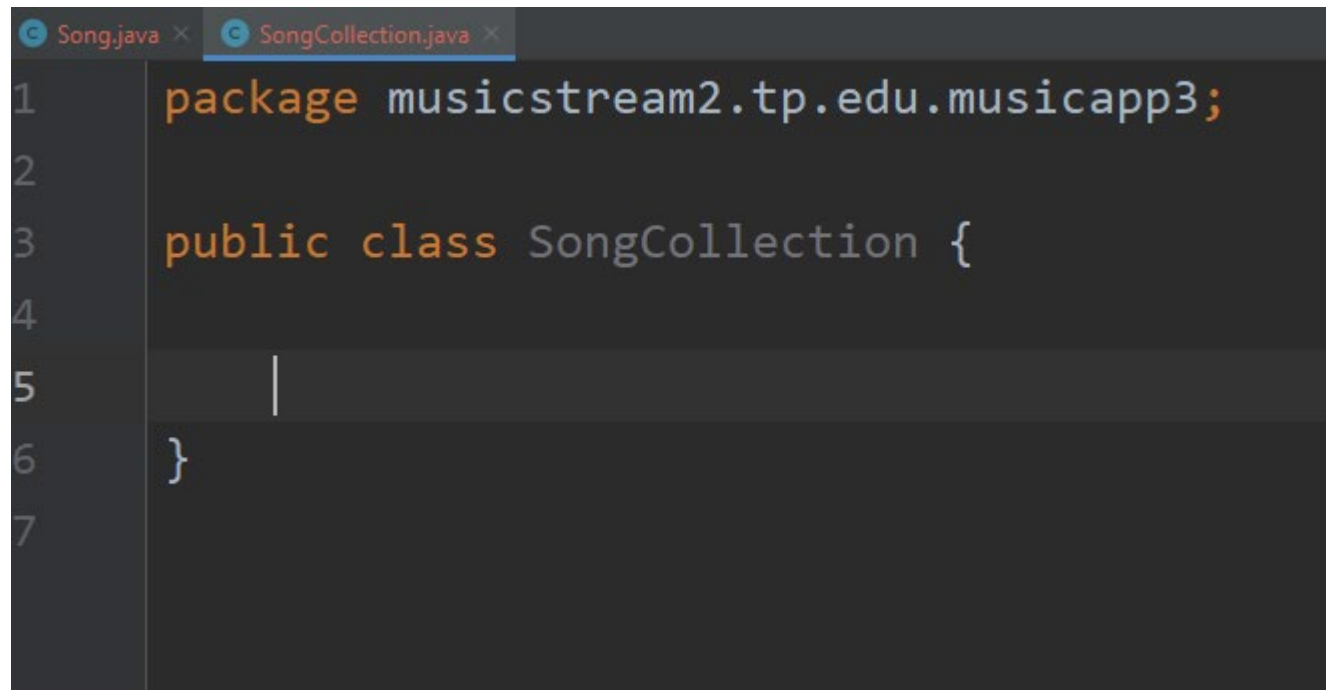
- Following the same steps in the previous exercise, create a new class under the `sg.edu.tp.musicstream` package folder and name it as `SongCollection`.
- The objective of creating the `SongCollection` object is to contain a collection of songs and provide various operations such as searching, and getting next and previous song.





# Creating a Song Collection Object

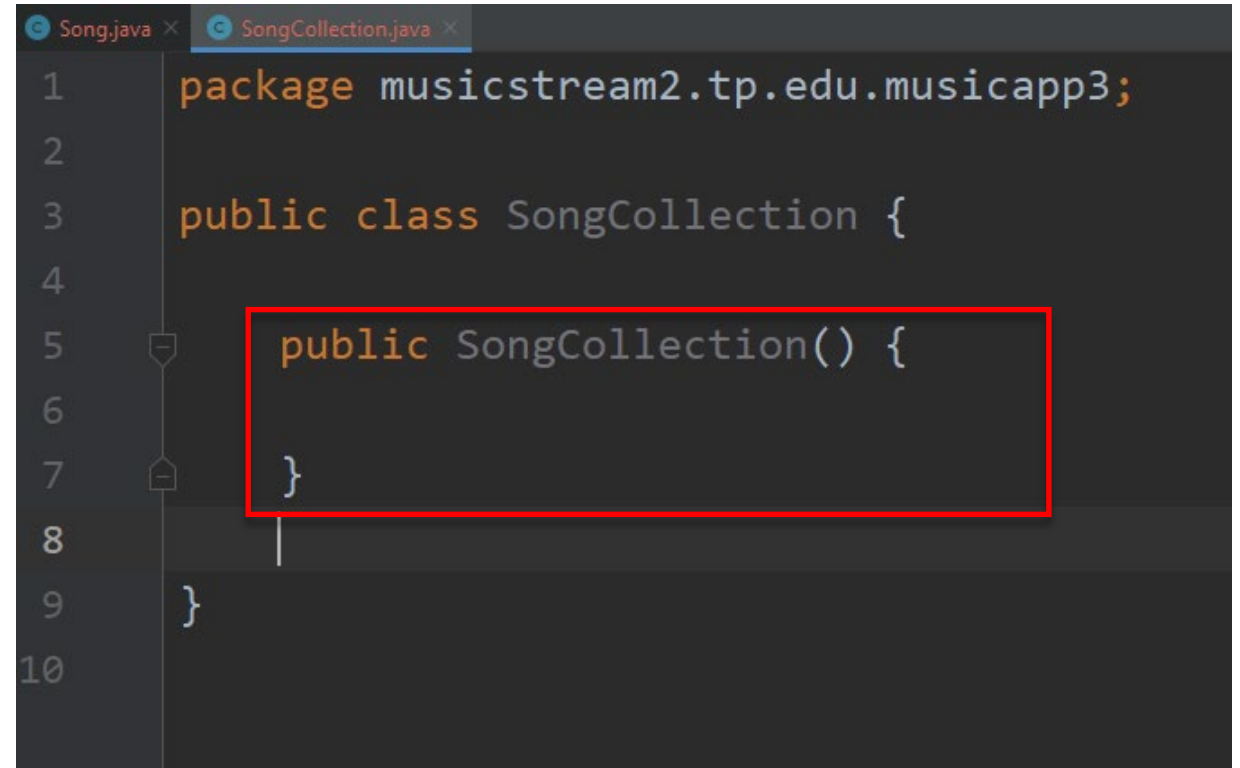
- Once the class is created, you will see a new Java file called `SongCollection` being created within the package folder `sg.edu.tp.musicstream`.
- Similarly, a few lines of code are also automatically generated as shown below.



```
1 package musicstream2.tp.edu.musicapp3;
2
3 public class SongCollection {
4
5     |
6 }
7
```

# Creating a Song Collection Object

- Let's add a *default constructor* into this class as shown here.
- We use the *default constructor* here without any parameters because we do not need to set any values to any instance of `SongCollection` object.



```
1 package musicstream2.tp.edu.musicapp3;
2
3 public class SongCollection {
4
5     public SongCollection() {
6
7     }
8
9 }
10
```

# Creating a Song Collection Object

- The next step is to create the songs in this collection.
- You can do so by running some codes in the constructor method since it will be called whenever the SongCollection class is being instantiated .
- In this method, you will use the **Song** object that you created previously to create a new song.
- **Let's use *The Way You Look Tonight*** as the first new song. (Your tutor will show you how to obtain the Spotify mp3 link and the corresponding information)

```
1 package musicstream2.tp.edu.musicapp3;
2
3 public class SongCollection {
4
5     public SongCollection() {
6         Song theWayYouLookTonight = new Song( id: "S1001",
7         title: "1. The Way You Look Tonight",
8         artiste: "Michael Buble",
9         fileLink: "https://p.scdn.co/mp3-preview/a5b8972e764025020625bbf9c1c2bbb06e394a60?cid=2afe87a64b0042dabf51f37318616965",
10        songLength: 4.66,
11        R.drawable.michael_buble_collection);
12     }
13
14 }
15
```

# Creating a Song Collection Object

Name of variable for a new song.

1) To create a new song object, it is important to use the **new** keyword followed by the object name with the value of each parameter if you are using the parameterized constructor.

2) The value of each property of the song must correspond to the parameters in the constructor of **Song.java**.

Type of object to create.

```
Song theWayYouLookTonight = new Song( id: "S1001",  
    title: "1. The Way You Look Tonight",  
    artiste: "Michael Buble",  
    fileLink: "https://p.scdn.co/mp3-preview/a5b8972e764025020625bbf9c1c2bbb06e394a60?cid=2afe87a64b0042dabf51f37318616965",  
    songLength: 4.66,  
    R.drawable.michael_buble_collection);  
}
```

```
public Song(String id, String title, String artiste, String fileLink, double songLength, int drawable){  
    this.id = id;  
    this.title = title;  
    this.artiste = artiste;  
    this.fileLink = fileLink;  
    this.songLength = songLength;  
    this.drawable = drawable;  
}
```

Constructor from **Song.java**.

To android, this represent a particular integer



# Try It Yourself!

- Now that you have created the first song. Try to create the second and third song yourself by following the example.
- The details of the songs can be found in the following website:
  - <http://comt.azurewebsites.net>
- Go to the above website and search for Michael Jackson. Select the album **Thriller** and look for the song Billie Jean.
- The variable name of the second song should be `billieJean`.
- Validate your finished codes with your lecturer.
- There are many other songs you can find in the above website to add to your collection!



---

---

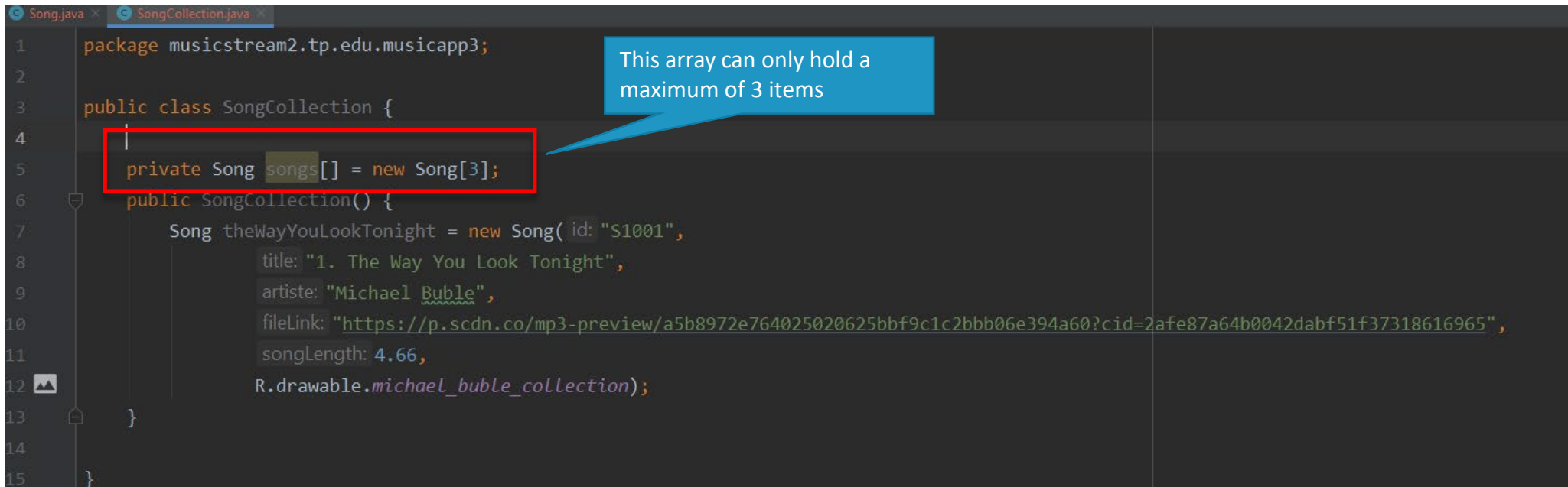
CREATE SONG LIST

---

---

# Creating Song List

- Back to the **MusicStream** project. After creating the songs, we still need to add the songs into a list.
- We can use Array to do so by creating an array of songs!
- Add the highlighted code for the array of **Song** as shown on the right. The variable name is **songs**.
- We place this line of code at this position because we need to use the **songs** variable within this class.



```
1 package musicstream2.tp.edu.musicapp3;
2
3 public class SongCollection {
4     |
5     private Song songs[] = new Song[3];
6     public SongCollection() {
7         Song theWayYouLookTonight = new Song( id: "S1001",
8             title: "1. The Way You Look Tonight",
9             artiste: "Michael Buble",
10             fileLink: "https://p.scdn.co/mp3-preview/a5b8972e764025020625bbf9c1c2bbb06e394a60?cid=2afe87a64b0042dabf51f37318616965",
11             songLength: 4.66,
12             R.drawable.michael_buble_collection);
13     }
14
15 }
```

# Creating Song List

- Next, we will add the three songs into the `songs` array as shown in the highlighted code.

Get the song details from [comt.azurewebsites.net](http://comt.azurewebsites.net)

Pushing each song into the songs array. Array starts from position 0

```
1 package sg.edu.tp.musicstream;
2
3 public class SongCollection {
4     private Song songs[] = new Song[3];
5
6     public SongCollection(){
7
8         Song theWayYouLookTonight = new Song( id: "S1001",
9             title: "1. The Way You Look Tonight",
10             artiste: "Michael Buble",
11             fileLink: "https://p.scdn.co/mp3-preview/a5b8972e764025020625bbf9c1c2bbb06e394a60?cid=2afe87a64b0042dabf51f37318616965",
12             songLength: 4.66,
13             R.drawable.michael_buble_collection);
14
15         Song billieJean = new Song( id: "S1002",
16             title: "2. Billie Jean",
17             artiste: "Michael Jackson",
18             fileLink: "https://p.scdn.co/mp3-preview/4eb779428d40d579f14d12a9daf98fc66c7d0be4?cid=2afe87a64b0042dabf51f37318616965",
19             songLength: 4.9,
20             R.drawable.billie_jean);
21
22         Song ed = new Song( id: "S1003",
23             title: "3. One",
24             artiste: "Ed Sheeran",
25             fileLink: "https://p.scdn.co/mp3-preview/daa8679253ba20620db6e1db9c88edfcf1f4069f?cid=2afe87a64b0042dabf51f37318616965",
26             songLength: 4.9,
27             R.drawable.photograph);
28
29         songs[0] = theWayYouLookTonight;
30         songs[1] = billieJean;
31         songs[2] = ed;
32     }
```

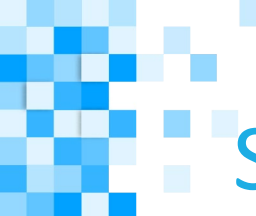




# Summary

---

- In the next lesson, you will learn how to create searching functionalities in the **SongCollection** object.



# Supplementary Materials

| Topic                    | Source                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
|--------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Variables and Data Types | <a href="http://www.learnjavaonline.org/en/Variables_and_Types">http://www.learnjavaonline.org/en/Variables_and_Types</a>                                                                                                                                                                                                                                                                                                                                                                                                      |
| Array                    | <a href="https://www.learnjavaonline.org/en/Arrays">https://www.learnjavaonline.org/en/Arrays</a><br><a href="https://www.javatpoint.com/array-in-java">https://www.javatpoint.com/array-in-java</a><br><a href="http://www.homeandlearn.co.uk/java/java_arrays.html">http://www.homeandlearn.co.uk/java/java_arrays.html</a><br><a href="https://www.lynda.com/Java-tutorials/Computer-Science-Principles-Lab-Java/508647-2.html">https://www.lynda.com/Java-tutorials/Computer-Science-Principles-Lab-Java/508647-2.html</a> |
| Objects                  | <a href="https://www.learnjavaonline.org/en/Objects">https://www.learnjavaonline.org/en/Objects</a><br><a href="https://www.infoq.com/articles/Teaching-Kids-Java">https://www.infoq.com/articles/Teaching-Kids-Java</a>                                                                                                                                                                                                                                                                                                       |

THE END



COMT